

SOURCE CODE LISTING

MEE2014 · Group 3 · B.Tech CSE (BCE) · 2026

SmartBin-1

ESP32 Firmware & Embedded Dashboard

Complete Arduino sketch SmartBin_Local.ino: ultrasonic ranging, LED + buzzer alerts, and a Wi-Fi access-point web dashboard that runs with no internet.

```
SmartBin_Local.ino - loop()

393 long duration = pulseIn(ECHO_PIN, HIGH, 30000);
394
395 if (duration == 0) {
396     distance_cm = BIN_DEPTH_CM;
397     fill_percentage = 0;
398 } else {
399     distance_cm = duration * 0.0343 / 2.0;
400
401     if (distance_cm > BIN_DEPTH_CM) distance_cm = BIN_DEPTH_CM;
402     if (distance_cm < 2.0) distance_cm = 2.0;
403
404     fill_percentage = round(((BIN_DEPTH_CM - distance_cm) / (BIN_DEPTH_CM - FULL_THRESHOLD_CM)) * 100.0);
405     fill_percentage = constrain(fill_percentage, 0, 100);
406 }
407
408 // 2. Hardware Output Control Logic
409 if (fill_percentage >= 80) {
410     digitalWrite(GREEN_LED_PIN, LOW);
411     digitalWrite(YELLOW_LED_PIN, LOW);
412     digitalWrite(RED_LED_PIN, HIGH);
413     tone(BUZZER_PIN, 2500);
```

430

LINES OF CODE

2

LIBRARIES

2

HTTP ENDPOINTS

6

GPIO PINS USED

SUBMITTED BY

Shivam Singh

Reg. No. 25BCE10736

Aryan Singh

Reg. No. 25BCE10798

VIT Bhopal University

School of Computer Science and Engineering (SCOPE)
Kotri Kalan, Ashta, Dist. Sehore - 466114, Madhya Pradesh, India

Prototype Unit V1.0

Board: ESP32 DevKit V1 · Arduino-ESP32 core

01 HARDWARE PIN MAP

Signal	GPIO	Dir	Wiring / role
HC-SR04 TRIG	GPIO5 (D5)	OUT	10 μ s trigger pulse starts each measurement
HC-SR04 ECHO	GPIO18 (D18)	IN	5 V echo reduced to 3.33 V by 1 k Ω / 2 k Ω divider
Green LED	GPIO14 (D14)	OUT	ON when fill < 50 % $\rightarrow$ OK
Yellow LED	GPIO27 (D27)	OUT	ON when fill 50 – 79 % $\rightarrow$ HALF FULL
Red LED	GPIO26 (D26)	OUT	ON when fill $\geq$ 80 % $\rightarrow$ FULL
Active buzzer	GPIO25 (D25)	OUT	tone(2500) while FULL; silenced otherwise

02 CONFIGURATION CONSTANTS

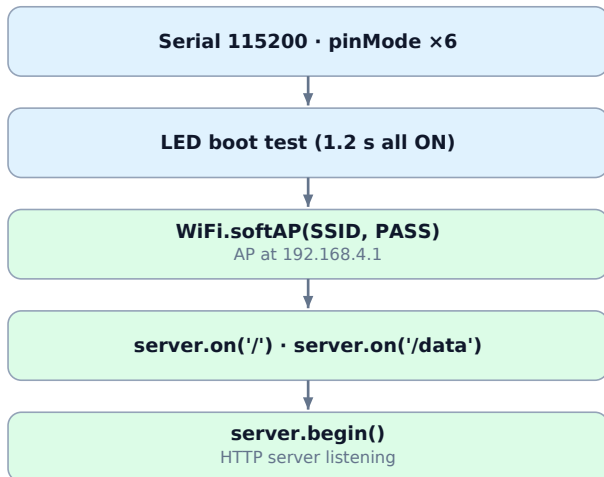
Name / setting	Value	Meaning
AP_SSID	"SmartBin-WiFi"	Name of the ESP32's own Wi-Fi hotspot
AP_PASS	"12345678"	WPA2 password (change before real deployment)
BIN_DEPTH_CM	14.4	Sensor-to-bottom distance of the empty bin (0 % fill)
FULL_THRESHOLD_CM	4.0	Sensor-to-waste distance at 100 % fill
Echo timeout	30000 μ s	pulseIn limit; a lost echo is treated as an empty bin
Distance clamp	2.0 – 14.4 cm	Reading is limited to this range before the % is computed
Alert thresholds	$\geq$ 80 / $\geq$ 50 %	FULL / HALF FULL (literals in loop())
Buzzer tone	2500 Hz	Played continuously while FULL
Loop delay / poll	200 ms / 500 ms	Sensing period / dashboard refresh interval

03 CODE MAP

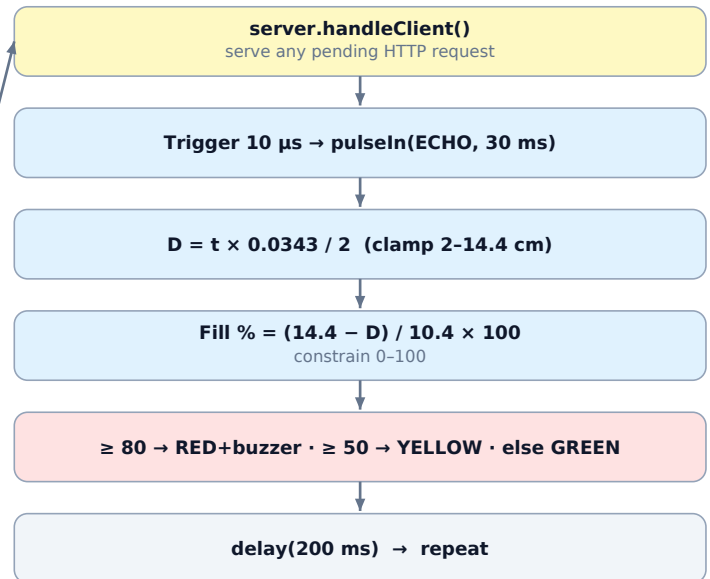
Section	Lines	Page	What it does
File header	1–9	4	Project, institution, group, authors, board
Includes	10–11	4	WiFi.h and WebServer.h (Arduino-ESP32 core)
Hotspot credentials	13–17	4	SSID and password of the access point
Pin mappings	19–26	4	GPIO assignments for sensor, LEDs, buzzer
Calibration + telemetry	29–39	4	Bin depth, full threshold, shared state variables
Dashboard · CSS	50–189	4	Dark card layout, bin graphic, status pill
Dashboard · HTML	191–250	6	Bin, metric tiles, priority panel, cleaning log
Dashboard · JavaScript	251–324	6	Polls /data every 500 ms, colours UI, logs cleaning
handleRoot / handleData	330–341	7	Serve the page (/) and JSON telemetry (/data)
setup()	343–381	7	Pins, LED boot test, start Wi-Fi AP + HTTP server
loop()	383–430	8	Measure, compute fill %, drive LEDs/buzzer, repeat

04 PROGRAM FLOW

setup() — runs once



loop() — repeats forever



05 STATE LOGIC

Fill level	Distance	Status string	LED	Buzzer	Dashboard colour · priority
< 50 %	> 9.2 cm	OK	Green (14)	OFF	Green · LOW
50 - 79 %	6.08 - 9.2 cm	HALF FULL	Yellow (27)	OFF	Yellow · MEDIUM
≥ 80 %	≤ 6.08 cm	FULL	Red (26)	2500 Hz	Red · HIGH (ACTION REQ.)

06 HTTP ENDPOINTS & DASHBOARD

Route	Returns	Details
GET /	text/html	Embedded dashboard stored in flash (PROGMEM raw string), no filesystem needed
GET /data	application/json	{"distance": 10.4, "fill": 39, "status": "OK"} — distance to 1 decimal, fill as integer %

Dashboard JavaScript polls /data every 500 ms and updates the bin graphic, distance and fill tiles, status pill and collection priority. A FULL → OK transition is auto-logged as “bin emptied”; the Mark-as-Cleaned button adds a manual entry. The log lives in the browser only.

07 BUILD, FLASH & USE

Arduino IDE → Tools → Board: ESP32 Dev Module · install the esp32 core (v2.x or v3.x) · pick the COM port.
 Upload; if the console shows “Connecting..._”, hold the BOOT button until writing starts.
 Serial Monitor at 115200 baud (press EN/RST) prints the hotspot banner and URL.
Phone: join Wi-Fi SmartBin-WiFi (mobile data off) → open <http://192.168.4.1>

Listing note: the printed code shows emoji as text tags — [green] [yellow] [siren] [pin] [broom] — because the mono font has no emoji glyphs. The real sketch uses the actual emoji characters.

SmartBin_Local.ino

Arduino-ESP32 · C++ with embedded HTML / CSS / JS · page 1 of 5

```

1 // =====
2 // Project: SmartBin Container Level Monitor (SmartBin-1)
3 // Mode: Pure Local Access Point (No External Wi-Fi Required)
4 // Institution: VIT BHOPAL UNIVERSITY
5 // Course/Group: MEE2014 - Group 3
6 // Authors: SHIVAM SINGH & ARYAN SINGH
7 // Board: ESP32 DevKit V1
8 // =====
9
10 #include <WiFi.h>
11 #include <WebServer.h>
12
13 // -----
14 // Local Standalone Hotspot Credentials
15 // -----
16 const char* AP_SSID = "SmartBin-WiFi";
17 const char* AP_PASS = "12345678";
18
19 // -----
20 // Hardware Pin Mappings
21 // -----
22 const int TRIG_PIN = 5; // GPIO5 / D5
23 const int ECHO_PIN = 18; // GPIO18 / D18 (via Voltage Divider)
24 const int GREEN_LED_PIN = 14; // GPIO14 / D14
25 const int YELLOW_LED_PIN = 27; // GPIO27 / D27
26 const int RED_LED_PIN = 26; // GPIO26 / D26
27 const int BUZZER_PIN = 25; // GPIO25 / D25
28
29 // -----
30 // Container Configuration (Calibrated 14.4cm Depth)
31 // -----
32 const float BIN_DEPTH_CM = 14.4; // Depth of empty bin
33 const float FULL_THRESHOLD_CM = 4.0; // Threshold distance for FULL state
34
35 // Telemetry Variables
36 float distance_cm = 14.4;
37 int fill_percentage = 0;
38 String status_msg = "OK [green]";
39
40 WebServer server(80);
41
42 // Embedded Web Dashboard Interface
43 const char HTML_DASHBOARD[] PROGMEM = R"rawliteral(
44 <!DOCTYPE html>
45 <html lang="en">
46 <head>
47 <meta charset="UTF-8">
48 <meta name="viewport" content="width=device-width, initial-scale=1.0">
49 <title>SmartBin-1 Control Center</title>
50 <style>
51 :root {
52 --bg-color: #0b1329;
53 --card-bg: #16203a;
54 --text-main: #f8fafc;
55 --text-muted: #94a3b8;
56 --accent-green: #22c55e;
57 --accent-yellow: #eab308;
58 --accent-red: #ef4444;
59 --border-color: #2a3756;
60 }
61 body {
62 font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
63 background: var(--bg-color);
64 color: var(--text-main);
65 margin: 0;
66 padding: 16px;
67 display: flex;
68 justify-content: center;
69 align-items: center;
70 min-height: 100vh;
71 box-sizing: border-box;
72 }
73 .app-card {
74 background: var(--card-bg);
75 width: 100%;
76 max-width: 480px;
77 padding: 24px;
78 border-radius: 24px;
79 box-shadow: 0 20px 40px rgba(0,0,0,0.6);
80 border: 1px solid var(--border-color);
81 }
82 .header { text-align: center; margin-bottom: 16px; }
83 .header h1 { font-size: 26px; margin: 0; color: #38bdf8; letter-spacing: 0.5px; }
84 .tag-badge {
85 display: inline-block;
86 background: #1e293b;
87 color: #cbd5e1;

```

DASHBOARD · HTML/CSS/JS

CSS

SmartBin_Local.ino

Arduino-ESP32 · C++ with embedded HTML / CSS / JS · page 2 of 5

```
88     font-size: 11px;
89     padding: 4px 10px;
90     border-radius: 12px;
91     margin-top: 6px;
92     font-weight: 600;
93     border: 1px solid var(--border-color);
94 }
95 .location-info {
96     font-size: 12px;
97     color: var(--text-muted);
98     margin-top: 6px;
99     font-weight: 500;
100 }
101 .bin-container {
102     width: 120px;
103     height: 190px;
104     border: 4px solid #475569;
105     border-radius: 0 0 18px 18px;
106     margin: 16px auto;
107     position: relative;
108     overflow: hidden;
109     background: #090d16;
110 }
111 .bin-fill-level {
112     width: 100%;
113     position: absolute;
114     bottom: 0;
115     height: 0%;
116     background: var(--accent-green);
117     transition: height 0.5s ease, background-color 0.5s ease;
118 }
119 .metrics-grid {
120     display: grid;
121     grid-template-columns: 1fr 1fr;
122     gap: 10px;
123     margin: 16px 0;
124 }
125 .metric-card {
126     background: #0d1527;
127     padding: 12px;
128     border-radius: 12px;
129     border: 1px solid var(--border-color);
130     text-align: center;
131 }
132 .metric-label { font-size: 10px; color: var(--text-muted); font-weight: 700; text-transform: uppercase; }
133 .metric-val { font-size: 20px; font-weight: 700; margin-top: 4px; }
134 .status-pill {
135     text-align: center;
136     padding: 8px 16px;
137     border-radius: 16px;
138     font-size: 15px;
139     font-weight: 700;
140     background: #1e293b;
141     margin: 10px 0;
142     border: 1px solid var(--border-color);
143 }
144 .info-box {
145     background: #0d1527;
146     border: 1px solid var(--border-color);
147     border-radius: 12px;
148     padding: 12px;
149     margin-top: 12px;
150     font-size: 12px;
151 }
152 .info-row { display: flex; justify-content: space-between; margin-bottom: 6px; }
153 .info-row:last-child { margin-bottom: 0; }
154 .history-log {
155     max-height: 90px;
156     overflow-y: auto;
157     background: #090d16;
158     border: 1px solid var(--border-color);
159     border-radius: 8px;
160     padding: 8px;
161     margin-top: 8px;
162     font-size: 11px;
163     color: #cbd5e1;
164 }
165 .history-item { padding: 3px 0; border-bottom: 1px solid #1e293b; }
166 .history-item:last-child { border-bottom: none; }
167 .clean-btn {
168     width: 100%;
169     background: #0284c7;
170     color: #fff;
171     border: none;
172     padding: 10px;
173     border-radius: 10px;
174     font-weight: 700;
```

SmartBin_Local.ino

Arduino-ESP32 · C++ with embedded HTML / CSS / JS · page 3 of 5

```
175     cursor: pointer;
176     margin-top: 10px;
177     transition: background 0.2s;
178   }
179   .clean-btn:active { background: #0369a1; }
180   .footer {
181     text-align: center;
182     margin-top: 18px;
183     padding-top: 12px;
184     border-top: 1px solid var(--border-color);
185     font-size: 11px;
186     color: var(--text-muted);
187   }
188   .authors { font-weight: 700; color: #38bdf8; margin-top: 4px; }
189 </style>
190 </head>
191 <body>
192
193 <div class="app-card">
194   <div class="header">
195     <h1>SmartBin-1</h1>
196     <div class="tag-badge">MEE2014 - GROUP 3</div>
197     <div class="location-info">[pin] VIT BHOPAL UNIVERSITY</div>
198   </div>
199
200   <div class="bin-container">
201     <div id="fillVisual" class="bin-fill-level"></div>
202   </div>
203
204   <div id="pillStatus" class="status-pill">STATUS: CONNECTED</div>
205
206   <div class="metrics-grid">
207     <div class="metric-card">
208       <div class="metric-label">Distance</div>
209       <div id="valDistance" class="metric-val">-- cm</div>
210     </div>
211     <div class="metric-card">
212       <div class="metric-label">Fill Level</div>
213       <div id="valFill" class="metric-val">-- %</div>
214     </div>
215   </div>
216
217   <!-- Dynamic Priority & Battery Panel -->
218   <div class="info-box">
219     <div class="info-row">
220       <span>Collection Priority:</span>
221       <strong id="valPriority" style="color: var(--accent-green);">LOW</strong>
222     </div>
223     <div class="info-row">
224       <span>Hardware Status:</span>
225       <strong style="color: var(--accent-green);">ONLINE (LOCAL AP)</strong>
226     </div>
227     <div class="info-row">
228       <span>Last Cleaned:</span>
229       <strong id="valLastCleaned">Not recorded yet</strong>
230     </div>
231   </div>
232
233   <!-- Cleaning Event History Log -->
234   <div class="info-box">
235     <div class="info-row">
236       <span><strong>Cleaning History Log:</strong></span>
237     </div>
238     <div id="historyLog" class="history-log">
239       <div class="history-item">System booted & monitoring initialized.</div>
240     </div>
241     <button class="clean-btn" onclick="logManualCleaning()">[broom] Mark as Cleaned Now</button>
242   </div>
243
244   <!-- Credits Footer -->
245   <div class="footer">
246     Copyright © 2026 SmartBin Systems
247     <div class="authors">SHIVAM SINGH & ARYAN SINGH</div>
248   </div>
249 </div>
250
251 <script>
252   let lastWasFull = false;
253   let lastCleanedTime = "On Initialization";
254
255   function updateTimestamp() {
256     const d = new Date();
257     return d.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit', second: '2-digit' });
258   }
259
260   function logManualCleaning() {
261     lastCleanedTime = updateTimestamp();
```

HTML

JavaScript

SmartBin_Local.ino

Arduino-ESP32 · C++ with embedded HTML / CSS / JS · page 4 of 5

```

262     document.getElementById('valLastCleaned').innerText = lastCleanedTime;
263
264     const log = document.getElementById('historyLog');
265     const item = document.createElement('div');
266     item.className = 'history-item';
267     item.style.color = 'var(--accent-green)';
268     item.innerText = `[${lastCleanedTime}] Manual cleaning verified.`;
269     log.prepend(item);
270 }
271
272 setInterval(async () => {
273     try {
274         const res = await fetch('/data');
275         const data = await res.json();
276
277         document.getElementById('valDistance').innerText = data.distance.toFixed(1) + ' cm';
278         document.getElementById('valFill').innerText = data.fill + ' %';
279
280         const visual = document.getElementById('fillVisual');
281         const pill = document.getElementById('pillStatus');
282         const priority = document.getElementById('valPriority');
283
284         visual.style.height = data.fill + '%';
285
286         if (data.fill >= 80) {
287             visual.style.backgroundColor = 'var(--accent-red)';
288             pill.innerText = 'STATUS: FULL [siren]';
289             pill.style.color = 'var(--accent-red)';
290             priority.innerText = 'HIGH (ACTION REQ.)';
291             priority.style.color = 'var(--accent-red)';
292             lastWasFull = true;
293         } else if (data.fill >= 50) {
294             visual.style.backgroundColor = 'var(--accent-yellow)';
295             pill.innerText = 'STATUS: HALF FULL [yellow]';
296             pill.style.color = 'var(--accent-yellow)';
297             priority.innerText = 'MEDIUM';
298             priority.style.color = 'var(--accent-yellow)';
299         } else {
300             visual.style.backgroundColor = 'var(--accent-green)';
301             pill.innerText = 'STATUS: OK [green]';
302             pill.style.color = 'var(--accent-green)';
303             priority.innerText = 'LOW';
304             priority.style.color = 'var(--accent-green)';
305
306             // Auto-detect cleaning cycle transition from FULL to OK
307             if (lastWasFull) {
308                 lastWasFull = false;
309                 lastCleanedTime = updateTimestamp();
310                 document.getElementById('valLastCleaned').innerText = lastCleanedTime;
311
312                 const log = document.getElementById('historyLog');
313                 const item = document.createElement('div');
314                 item.className = 'history-item';
315                 item.style.color = 'var(--accent-green)';
316                 item.innerText = `[${lastCleanedTime}] Auto-Detected: Bin emptied by staff.`;
317                 log.prepend(item);
318             }
319         }
320     } catch (e) {
321         console.log('Polling telemetry...');
322     }
323 }, 500);
324 </script>
325
326 </body>
327 </html>
328 )rawliteral";
329
330 void handleRoot() {                                     handleRoot()
331     server.send(200, "text/html", HTML_DASHBOARD);
332 }
333
334 void handleData() {                                     handleData()
335     String json = "{";
336     json += "\"distance\":" + String(distance_cm, 1) + ",";
337     json += "\"fill\":" + String(fill_percentage) + ",";
338     json += "\"status\":" + status_msg + "\"";
339     json += "}";
340     server.send(200, "application/json", json);
341 }
342
343 void setup() {                                         setup()
344     Serial.begin(115200);
345     delay(1000);
346
347     // Configure Pins
348     pinMode(TRIG_PIN, OUTPUT);

```

SmartBin_Local.ino

Arduino-ESP32 · C++ with embedded HTML / CSS / JS · page 5 of 5

```

349  pinMode(ECHO_PIN, INPUT);
350  pinMode(GREEN_LED_PIN, OUTPUT);
351  pinMode(YELLOW_LED_PIN, OUTPUT);
352  pinMode(RED_LED_PIN, OUTPUT);
353  pinMode(BUZZER_PIN, OUTPUT);
354
355  // Hardware Boot Test
356  digitalWrite(GREEN_LED_PIN, HIGH);
357  digitalWrite(YELLOW_LED_PIN, HIGH);
358  digitalWrite(RED_LED_PIN, HIGH);
359  delay(1200);
360
361  digitalWrite(GREEN_LED_PIN, LOW);
362  digitalWrite(YELLOW_LED_PIN, LOW);
363  digitalWrite(RED_LED_PIN, LOW);
364  noTone(BUZZER_PIN);
365
366  // Directly Start Local Access Point Mode
367  WiFi.mode(WIFI_AP);
368  WiFi.softAP(AP_SSID, AP_PASS);
369
370  Serial.println("\n=====");
371  Serial.println(" SmartBin Standalone Hotspot Active ");
372  Serial.println(" Connect Phone Wi-Fi to: SmartBin-WiFi");
373  Serial.println(" Password: " + String(AP_PASS));
374  Serial.print(" Open Browser URL: http://");
375  Serial.println(WiFi.softAPIP()); // http://192.168.4.1
376  Serial.println("=====");
377
378  server.on("/", handleRoot);
379  server.on("/data", handleData);
380  server.begin();
381 }
382
383 void loop() {
384     server.handleClient();
385
386     // 1. Measure Distance via Ultrasonic Sensor
387     digitalWrite(TRIG_PIN, LOW);
388     delayMicroseconds(2);
389     digitalWrite(TRIG_PIN, HIGH);
390     delayMicroseconds(10);
391     digitalWrite(TRIG_PIN, LOW);
392
393     long duration = pulseIn(ECHO_PIN, HIGH, 30000);
394
395     if (duration == 0) {
396         distance_cm = BIN_DEPTH_CM;
397         fill_percentage = 0;
398     } else {
399         distance_cm = duration * 0.0343 / 2.0;
400
401         if (distance_cm > BIN_DEPTH_CM) distance_cm = BIN_DEPTH_CM;
402         if (distance_cm < 2.0) distance_cm = 2.0;
403
404         fill_percentage = round(((BIN_DEPTH_CM - distance_cm) / (BIN_DEPTH_CM - FULL_THRESHOLD_CM)) * 100.0);
405         fill_percentage = constrain(fill_percentage, 0, 100);
406     }
407
408     // 2. Hardware Output Control Logic
409     if (fill_percentage >= 80) {
410         digitalWrite(GREEN_LED_PIN, LOW);
411         digitalWrite(YELLOW_LED_PIN, LOW);
412         digitalWrite(RED_LED_PIN, HIGH);
413         tone(BUZZER_PIN, 2500);
414         status_msg = "FULL [siren]";
415     } else if (fill_percentage >= 50) {
416         digitalWrite(GREEN_LED_PIN, LOW);
417         digitalWrite(YELLOW_LED_PIN, HIGH);
418         digitalWrite(RED_LED_PIN, LOW);
419         noTone(BUZZER_PIN);
420         status_msg = "HALF FULL [yellow]";
421     } else {
422         digitalWrite(GREEN_LED_PIN, HIGH);
423         digitalWrite(YELLOW_LED_PIN, LOW);
424         digitalWrite(RED_LED_PIN, LOW);
425         noTone(BUZZER_PIN);
426         status_msg = "OK [green]";
427     }
428
429     delay(200);
430 }

```

loop()

ranging

alerts